

NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF APPLIED SCIENCES

DEPARTMENT OF INFORMATICS AND ANALYTICS

GROUP NAME: GROUP 5

COURSE NAME: Statistical Programming

COURSE CODE: SCI2105

ASSIGNMENT NUMBER: GROUP ASSIGNMENT

Group members

RICHARDSON RWATIRERA N02216814Y

ROMEO SIBANDA N0239640T

JUSTINE RUPANGA N0237199G

LESLIE NDEBELE N0230634T

THABANI DLAMINI N0232266G

NYSHA ZWANDAZIIWAH N0233867Q

SIMARASHE MATENGAH N0232801P

TADIWANASHE MUNARWAO N0231112J

TADIWANASHE CHUMA N02310849G

TATENDA MUTYAMBIZI N0238030H

1. State and explain any 5 Data Structures that are available in R Language

1. Vectors

Vectors are one-dimensional arrays of elements of the same data type. They can be numeric, character, or logical. Vectors are the most basic data structure in R.

Example:

```
x <- c(1, 2, 3, 4, 5)

x
```

2. Lists

Lists are collections of objects of different data types, including vectors, matrices, data frames, and other lists. Lists are denoted by the list() function.

Example:

```
x <- list(1, "hello", TRUE, c(1, 2, 3))

x
```

3. Matrices

Matrices are two-dimensional arrays of elements of the same data type. They are denoted by the matrix() function.

Example:

```
x <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2)

x
```

4. Data Frames

Data frames are two-dimensional arrays of elements of different data types, similar to matrices. However, each column can have a different data type. Data frames are denoted by the data.frame() function.

Example:

```
x <- data.frame(name = c("John", "Mary", "David"), age = c(25, 31, 42))

x
```

5. Arrays

Arrays are multi-dimensional arrays of elements of the same data type. They are denoted by the array() function.

Example:

```
x <- array(c(1, 2, 3, 4, 5, 6), dim = c(2, 3))
```

x

2. Discuss the main features of R programming language 10 marks

1. Statistical Analysis: R is specifically designed for statistical analysis, providing an extensive range of statistical techniques, including hypothesis testing, confidence intervals, regression analysis, time series analysis, and more.

2. Data Visualization: R provides a wide range of data visualization tools, including plots, charts, and graphs, to help users understand and communicate complex data insights.

3. Programming Language: R is a full-fledged programming language, allowing users to write custom functions, loops, and conditional statements to automate tasks and perform complex analyses.

4. Packages and Libraries: R has a vast collection of packages and libraries that provide additional functionality, including data manipulation, machine learning, and data visualization.

5. Cross-Platform Compatibility: R is available on multiple platforms, including Windows, macOS, and Linux, making it a versatile tool for data analysis.

6. Open-Source: R is an open-source language, which means that it is free to use, modify, and distribute.

7. Large Community: R has a large and active community of users, developers, and contributors, providing extensive support, documentation, and resources.

8. Integration with Other Tools: R can be easily integrated with other tools and languages, such as Python, SQL, and Excel, making it a versatile tool for data analysis.

9. Dynamic Graphics: R provides dynamic graphics capabilities, allowing users to create interactive and dynamic visualizations.

10. Extensive Documentation: R has extensive documentation, including manuals, tutorials, and online resources, making it easy for users to learn and use the language.

3. Write a program in R to find the factorial of a number (use recursion).

```r
facto<-function(a){
  if(a==1 | a==0){
    return(1)
  }else{
    return(a*facto(a-1))
  }
}


print(facto(5))
```

4. Write R function to check whether the given number is prime or not.

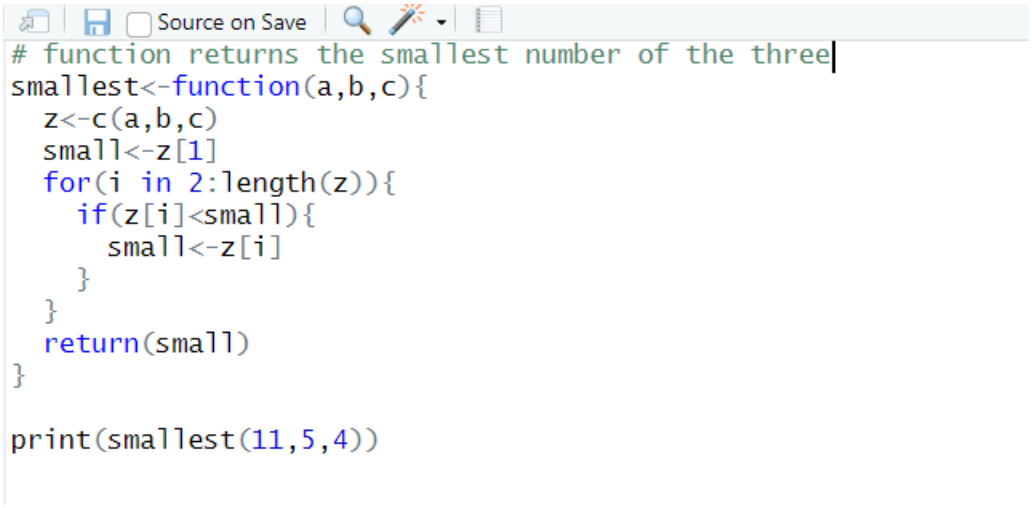
```r
#function to check whether the given number is prime or not

is_prime <- function(n) {
  if(n<2){
    print(paste("The number",n,"is not a prime number"))
  }
  if(n==2){
    print(paste("The number",n,"is a prime number"))
  }
  if(n>2){
    prime=TRUE
    for(j in 2:sqrt(n)){
      if(n%%j==0){
        prime=FALSE
        break
      }
    }
    if(prime==TRUE){
      print(paste("The number",n,"is a prime number"))
    }else{
      print(paste("The number",n,"is not a prime number"))
    }
  }

}

is_prime(5)
```

5. Write a function that takes 3 numbers *a*, *b*, and *c* as inputs and returns the smallest number of the three.



```
# function returns the smallest number of the three
smallest<-function(a,b,c){
  z<-c(a,b,c)
  small<-z[1]
  for(i in 2:length(z)){
    if(z[i]<small){
      small<-z[i]
    }
  }
  return(small)
}

print(smallest(11,5,4))
```

6. Show how variables are declared and explain why it is necessary to declare variables in programming

how variables are declared in R:

#Numeric variable

x <- 5

#Character variable

name <- "John"

#Logical variable

isAdmin <- TRUE

#Vector variable

scores <- c(90, 85, 95)

Declaring variables is necessary in programming because it enables the programming language to allocate memory space for that variable which ensures that the variable has enough space to store its value.

It is also necessary because it specifies its data type, which helps the programming language understand how to store and manipulate the variable's value.

7. State and explain 5 basic data types used with R programming

Numeric Data Type

The numeric data type in R represents numbers, including integers and decimals. This data type is used for mathematical calculations, such as addition, subtraction, multiplication, and division. For

example, x <- 5 or y <- 3.14 are numeric values. The numeric data type is also used for statistical calculations, like mean, median, and standard deviation.

Character Data Type

The character data type in R represents text strings, including letters, numbers, and special characters. This data type is used for storing and manipulating text data, such as names, addresses, comments, and descriptions. For example, name <- "John" or sentence <- "Hello, World!" are character values.

Logical Data Type

The logical data type in R represents Boolean values, which can be either TRUE or FALSE. This data type is used for making decisions, filtering data, and performing conditional operations. For example, isAdmin <- TRUE or isStudent <- FALSE are logical values. The logical data type is also used for validating data and checking conditions.

Integer Data Type

The integer data type in R represents whole numbers, without decimals. This data type is used for storing and manipulating whole numbers, such as ages, counts, indices, and quantities. For example, age <- 25 or count <- 10 are integer values.

Complex Data Type

In R, complex data types are used to represent complex numbers, which are numbers that have both real and imaginary parts. You can create a complex value using the 1+2i syntax, e.g., z <- 3 + 4i. Alternatively, use the complex() function, e.g., complex(real = 3, imaginary = 4).

8. State and discuss 5 functions which can be used for debugging in R

1. traceback()

The traceback() function is used to display the call stack after an error has occurred. When an error is encountered, calling traceback() will show you the sequence of function calls that led to the error, which can help you pinpoint where things went wrong.

Usage:

```
#Example usage after an error occurs

my_function <- function() {

  stop("An error occurred!")

}

my_function()  # This will trigger an error

traceback()    # Call this immediately after the error to see the call stack
```

2. debug()

The debug() function allows you to step through a function line by line. When you call debug() on a function, R will enter a debugging environment when that function is called, allowing you to inspect variables and control the flow of execution.

Usage:

```
my_function <- function(x) {
 y <- x + 1
 stop("An error occurred!")
 return(y)
}


debug(my_function)  # Set the function to debug
my_function(5)     # This will enter the debug mode
```

3. browser()

The browser() function can be inserted directly into your code to create a breakpoint. When R reaches the browser() call, it will pause execution and allow you to inspect the current environment, including variable values.

Usage:

```
my_function <- function(x) {
 y <- x + 1
 browser()  # Execution will pause here
 return(y)
}
my_function(5)  # This will pause at the browser() line
```

4. undebug()

The undebug() function is used to remove the debugging flag from a function that was previously set with debug(). This is useful when you no longer need to step through the function and want to restore its normal behavior.

Usage:

```
debug(my_function)  # Set the function to debug
undebug(my_function)  # Remove debugging from the function
```

5. recover()

The recover() function is used to change the error handling behavior in R. When an error occurs, instead of stopping execution, recover() allows you to choose which frame of the call stack to inspect. This can be particularly useful for debugging complex functions.

Usage:

Example usage

```
options(error = recover)  # Set recover as the error handler

my_function <- function() {

  stop("An error occurred!")

}

my_function()  # When this error occurs, you can choose to inspect the call stack
```

9. What is Statistical Inference

Statistical Inference is the process of using sample data to make conclusions or predictions about a larger population. It involves using statistical methods to analyze data and make inferences about the population parameters.

10. What packages are used for data mining in R?

1. dplyr: This package is part of the tidyverse and is essential for data manipulation. It provides a set of functions for data wrangling, making it easier to filter, select, and summarize data.

2. ggplot2: Also part of the tidyverse, ggplot2 is used for data visualization. It allows you to create complex and informative graphics, which are crucial for exploring data and identifying patterns.

3. caret: The caret package (short for Classification And REgression Training) streamlines the process of creating predictive models. It provides tools for data splitting, pre-processing, feature selection, and model tuning.

4. randomForest: This package implements the random forest algorithm, which is useful for both classification and regression tasks. It is particularly effective for handling large datasets with many variables.

5. rpart: The rpart package is used for recursive partitioning and regression trees. It helps in building decision trees for classification and regression problems.

6. e1071: This package includes functions for various machine learning techniques, including support vector machines (SVM), naive Bayes, and clustering methods.

11. How would you make multiple plots onto a single page in R?

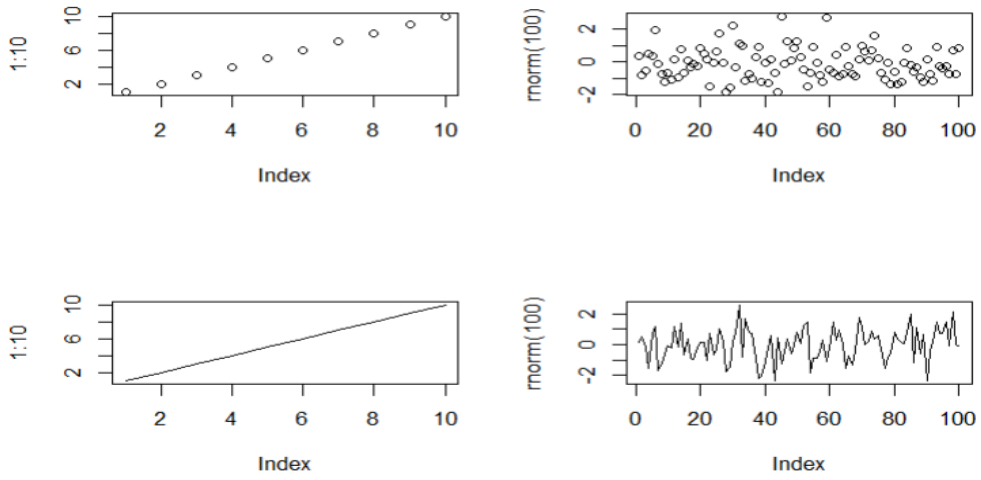
I can use the par() function or the gridExtra package to create multiple plots on a single page in R.

Method 1: Using par() function

```r
#Create a 2x2 plot grid
par(mfrow = c(2, 2))

#Create four plots
plot(1:10)
plot(rnorm(100))
plot(1:10, type = "l")
plot(rnorm(100), type = "l")
```

Output:



Method 2: Using ggplot2 package

```r
#Load ggplot2 library
library(ggplot2)

#Create four plots
p1 <- ggplot(mtcars, aes(x = wt, y = mpg)) + geom_point()
p2 <- ggplot(mtcars, aes(x = wt, y = mpg)) + geom_line()
p3 <- ggplot(mtcars, aes(x = wt, y = mpg)) + geom_bar(stat = "identity")
p4 <- ggplot(mtcars, aes(x = wt, y = mpg)) + geom_smooth()
#Use grid.arrange() to arrange plots
library(gridExtra)
grid.arrange(p1, p2, p3, p4, nrow = 2)
```

12. Write a custom function which will replace all the missing values in a vector with the mean of values.

```r
# function replace all the missing values in a vector with the mean of values.
replace<-function(x){
  mean_value<-mean(x,na.rm=TRUE)
  x[is.na(x)==TRUE]<-mean_value
  return(x)
}

z<-c(1,NA,5,NA,20)

print(replace(z))
```

13. How would you find the number of missing values in a dataset and remove all of them?

To find the number of missing values in a dataset we can use sum() and is.na()

For example:

```r
# read and load the dataset
x=read.csv("dataset1.csv")

# Count the number of missing values using sum() function and is.na() function.
missing_values=sum(is.na(x)==TRUE)

# Display the number of missing values
print(paste("Number of missing values:",missing_values,"values"))
```

To remove the missing values in a dataset we can use na.omit() function

For example:

```
Source on Save
# read and load the dataset
df=read.csv("dataset1.csv")

# remove the missing values
df=na.omit(df)

# display the dataset
print(df)
```

14. What is correlation? How would you measure correlation in R

Correlation is a statistical measure that describes the relationship between two continuous variables. It measures how strongly the variables tend to change together. Correlation is usually measured using the correlation coefficient, which ranges from -1 (perfect negative correlation) to 1 (perfect positive correlation).

To measure correlation in R, you can use the following methods:

1. Correlation coefficient (corrcoef): The cor() function calculates the correlation coefficient between two vectors.

x <- c(1, 2, 3, 4, 5)

y <- c(2, 4, 6, 8, 10)

cor(x, y)

2. Pearson's correlation coefficient: Pearson's correlation coefficient measures the linear relationship between two continuous variables. You can calculate it using the cor() function with the method = "pearson" argument.

cor(x, y, method = "pearson")

3. Spearman's rank correlation coefficient: Spearman's rank correlation coefficient measures the monotonic relationship between two continuous variables. You can calculate it using the cor() function with the method = "spearman" argument.

cor(x, y, method = "spearman")

15. What is Rmarkdown? What is the use of it?

R Markdown is a file format that combines Markdown text, R code, and executable code chunks to generate dynamic documents, reports, and presentations (Allaire et al., 2022). It's a powerful tool for creating reproducible and shareable documents that showcase data analysis, visualization, and insights. The primary uses of R Markdown include:

1. Dynamic reporting: Create reports that automatically update when data changes.

2. Data storytelling: Combine text, code, and visualizations to tell compelling stories with data (Xie, 2015).

3. Reproducible research: Share research findings, including data, code, and results, to facilitate reproducibility and collaboration.

4. Education and training: Create interactive tutorials, workshops, and courses that integrate code, examples, and exercises.

5. Business intelligence: Generate reports, dashboards, and presentations that provide insights into business data and metrics.

6. Academic publishing: Submit papers and articles that include executable code, data, and results to facilitate peer review and reproducibility.

7. Personal projects and blogging: Create and share personal projects, such as data visualizations, analyses, or tutorials, using R Markdown.

16. How do you install a package in R?

To install a package in R, you can use the install.packages() function. Here's how you do it:

1. **Open your R environment** (like RStudio, R console, or Jupyter notebook with an R kernel).
2. Use the following command to install the desired package. For example, to install the package ggplot2:

   install.packages("ggplot2")

3. Press Enter to execute the command. R will download and install the package from CRAN (the Comprehensive R Archive Network).

17. What are the steps to build and evaluate a linear regression model in R?

1. Data Exploration

Begin by loading your dataset and exploring its structure. Use functions like str(), summary(), and head() to understand the variables and their distributions. You should also Visualize relationships between the dependent variable and independent variables using scatter plots or pair plots.

Example

```r
#Load necessary libraries
library(ggplot2)
#Load your dataset
data <- read.csv("your_data.csv")

#Explore the data
str(data)
summary(data)
ggplot(data, aes(x = independent_var, y = dependent_var)) + geom_point()
```

2. Data Preprocessing

In step 2, You must handle missing values, outliers, and categorical variables. This usually involves imputing missing values or remove outliers depending on your analysis. You may convert categorical variables into factors if necessary.

Example:

```r
#Example of handling missing values
data <- na.omit(data)
#Convert categorical variables
data$category_var <- as.factor(data$category_var)
```

3. Split the data into training and testing sets*

Split your data into training (70-80%) and testing sets (20-30%) using the set.seed() and sample.split()functions. This a crucial step that will enable us to train our model.

Example

```
Source on Save
#Load the data
data(mtcars)

#Split the data into training and testing sets
set.seed(123)
train_idx<-sample.split(mtcars$mpg, SplitRatio=0.7)
train_data<-mtcars[train_idx,]
test_data<-mtcars[!train_idx,]
```

4. Fitting the Model

In this step, Use the lm() function to build the linear regression model on the training data.We are now building our functional linear regression model.

Example:

```
#Fit the linear regression on the train data model
model<-lm(mpg~wt,data=train_data)

summary(model)
```

5. Checking Assumptions

Validate the assumptions of linear regression, including linearity, independence, homoscedasticity, and normality of residuals. You can use diagnostic plots to assess these assumptions.

Example

```
#Diagnostic plots
par(mfrow = c(2, 2))
plot(model)
```

6. Evaluating the Model

We now need to evaluate the model so that we may know how well it is perfoming. To do this, use the summary() function to obtain details about the model, including coefficients, R-squared value, and p-values for the predictors. You must also calculate additional metrics such as Mean Squared Error (MSE) or Root Mean Squared Error (RMSE) to evaluate model performance.Such metrics clearly show how well the model is performimg

Example:

```
#Model summary
summary(model)

#Calculate RMSE
predictions <- predict(model, data)
rmse <- sqrt(mean((data$dependent_var - predictions)^2))
```

7. Making Predictions

Once the model is validated and is performing at peak performance , you can use it to make predictions on new data.

Example:

```
#Making predictions on new data

new_data <- data.frame(independent_var1 = c(value1), independent_var2 = c(value2))

predicted_values <- predict(model, new_data)
```

18. Name 6 packages in R, which can be used for data imputation

1. mice: This package implements multiple imputation using chained equations, allowing for flexible handling of different types of variables and missing data patterns.

2. missForest: A non-parametric imputation method that uses random forests to predict missing values, making it effective for mixed-type data.

3. Amelia: This package provides a fast and flexible approach to multiple imputation, particularly useful for time-series data and datasets with a lot of missing values.

4. Hmisc: Offers various functions for data analysis, including imputation methods like predictive mean matching and regression-based imputation.

5. mi: This package focuses on multiple imputation and provides tools for handling missing data in a variety of contexts, including longitudinal data.

6. VIM: Primarily used for visualizing missing data, it also includes functions for imputation, making it a useful tool for exploratory data analysis alongside imputation.

19. What is a Random Forest? How do you build and evaluate a Random Forest in R?

A Random Forest is an ensemble learning method that combines multiple decision trees to improve the accuracy and robustness of predictions. It operates by constructing multiple decision trees during training and outputs the mode of the classes (for classification) or the mean prediction (for regression) of the individual trees.

To build and evaluate a Random Forest in R, one must follow the below steps:

1. Install and Load Necessary Packages

Make sure you have the randomForest package installed and loaded. Use the code below to install and load the package.

```
#Install the package if you haven't already
install.packages("randomForest")

#Load the package
library(randomForest)
```

2. Prepare Your Data

Load your dataset and split it into training and testing sets. Ensure that your data is clean and preprocessed (e.g., handling missing values, encoding categorical variables).

Example:

```
#Load the data
data(mtcars)

#remove missing values
mtcars=na.omit(mtcars)

#Split the data into training and testing sets
set.seed(123)
train_idx<-sample(1:nrow(mtcars),0.7,nrow(mtcars))
train_data<-mtcars[train_idx,]
test_data<-mtcars[-train_idx,]
```

3. Build the Random Forest Model

Use the randomForest() function to create the model. Specify the formula and the dataset.

Example:

```r
#Build the Random Forest model

rf_model <- randomForest(dependent_var ~ ., data = train_data, ntree = 100)
```

4. Evaluate the Model

After building the model, you can evaluate its performance on the test set. Use the predict() function to make predictions and then compare them to the actual values.

```r
#Make predictions on the test set
predictions <- predict(rf_model, test_data)

#Evaluate the model's performance
confusion_matrix <- table(test_data$dependent_var, predictions)
print(confusion_matrix)

#Calculate accuracy
accuracy <- sum(diag(confusion_matrix)) / sum(confusion_matrix)
print(paste("Accuracy:", round(accuracy, 2)))
```

20. Explain about confusion matrix in R?

A confusion matrix is a performance measurement tool for classification models, showing the counts of actual vs. predicted classifications. It helps assess the accuracy of a model and its ability to classify data correctly in R.

Example of creating a confusion matrix in R:

```r
#Load the necessary library
library(caret)

#Create a predicted outcome
predicted <- factor(c("A", "B", "A", "B", "A", "B", "A", "A"))

#Create an actual outcome
actual <- factor(c("A", "B", "A", "A", "B", "B", "A", "B"))

#Create a confusion matrix
confusion_matrix <- table(actual, predicted)

#Print the confusion matrix
print(confusion_matrix)
```

21. Tell me something about shinyR.

Shiny (also known as **Shiny R**) is an R package that allows you to build interactive web applications directly from R. It is particularly useful for creating data-driven dashboards, visualizations, and user interfaces that interactively display and manipulate data. With Shiny, you can take the results of your R analysis and turn them into interactive web applications without needing any knowledge of HTML, CSS, or JavaScript.

22. What is T-Test in R Programming?

In R programming, a **T-test** is a statistical test used to compare the means of two groups to determine if they are significantly different from each other. It is commonly used to test hypotheses about population means when the data follows a normal distribution, and it helps to evaluate whether the observed differences between the groups are statistically significant.

23. What packages are used for data mining in R?

1. dplyr: This package is part of the tidyverse and is essential for data manipulation. It provides a set of functions for data wrangling, making it easier to filter, select, and summarize data.

2. ggplot2: Also part of the tidyverse, ggplot2 is used for data visualization. It allows you to create complex and informative graphics, which are crucial for exploring data and identifying patterns.

3. caret: The caret package (short for Classification And REgression Training) streamlines the process of creating predictive models. It provides tools for data splitting, pre-processing, feature selection, and model tuning.

4. randomForest: This package implements the random forest algorithm, which is useful for both classification and regression tasks. It is particularly effective for handling large datasets with many variables.

5. rpart: The rpart package is used for recursive partitioning and regression trees. It helps in building decision trees for classification and regression problems.

6. e1071: This package includes functions for various machine learning techniques, including support vector machines (SVM), naive Bayes, and clustering methods.

24. Give examples of while and for loop in R.

Example of a while loop in R:

```
i<-1
#Set the condition for the while loop
while(i<5){
  print(i)
  i<-i+1
}
```

Example of a for loop in R:

```
number<-1:5
#Use a for loop to iterate over the sequence
for(i in numbers){
  Print(i)
}
```

25.What do you know about the rattle package in R?

The rattle package in R is a graphical user interface (GUI) for data mining and machine learning tasks. It is built on top of other popular R packages and provides a user-friendly interface for performing various data analysis and modeling tasks without requiring users to write code manually.

26. Explain about "initialize()" function in R?

The initialize() function in R is primarily used in object-oriented programming (OOP) contexts, particularly with S3 and S4 classes. It is part of the S4 class system in R and is used to initialize an object when it is created, setting the initial values for its slots (attributes or components).

The initialize() function is used to set the initial values of an object when it is instantiated from a class. It is automatically called when you create an instance of an S4 class using the new() function.

27. Name some functions which can be used for debugging in R?

1.debug()

2.traceback()

3.brower()

4.recover()

5.undebug()

28. What is clustering?

Clustering is a type of unsupervised machine learning technique used to group a set of objects (data points) into clusters or groups, such that objects within the same group are more similar to each other than to those in other groups. The goal of clustering is to identify inherent structures or patterns in data without the need for labeled data.

29. Write R code to generate first n terms of a Fibonacci series

```
# function to generate first n terms of a Fibonacci series
fibonaci<-function(n){
  fib_series<-c(0,1)
  for(i in 3:n){
    fib_series[i]<-fib_series[i-1]+fib_series[i-2]
  }
  return(fib_series)
}

n<-10
print(fibonaci(n))
```

30. List the data structures in R with examples.

1. Vectors

Example:

x <- c(1, 2, 3, 4, 5)

x

2. Lists

Example:

x <- list(1, "hello", TRUE, c(1, 2, 3))

x

3. Matrices

Example:

```
x <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2)

x
```

4. Data Frames

Example:

```
x <- data.frame(name = c("John", "Mary", "David"), age = c(25, 31, 42))

x
```

5. Arrays

Example:

```
x <- array(c(1, 2, 3, 4, 5, 6), dim = c(2, 3))

x
```

31. Explain in details all the available datatypes in R

Numeric Data Type

The numeric data type in R represents numbers, including integers and decimals. This data type is used for mathematical calculations, such as addition, subtraction, multiplication, and division. For example, x <- 5 or y <- 3.14 are numeric values. The numeric data type is also used for statistical calculations, like mean, median, and standard deviation.

Character Data Type

The character data type in R represents text strings, including letters, numbers, and special characters. This data type is used for storing and manipulating text data, such as names, addresses, comments, and descriptions. For example, name <- "John" or sentence <- "Hello, World!" are character values.

Logical Data Type

The logical data type in R represents Boolean values, which can be either TRUE or FALSE. This data type is used for making decisions, filtering data, and performing conditional operations. For example, isAdmin <- TRUE or isStudent <- FALSE are logical values. The logical data type is also used for validating data and checking conditions.

Integer Data Type

The integer data type in R represents whole numbers, without decimals. This data type is used for storing and manipulating whole numbers, such as ages, counts, indices, and quantities. For example, age <- 25 or count <- 10 are integer values.

Complex Data Type

In R, complex data types are used to represent complex numbers, which are numbers that have both real and imaginary parts. You can create a complex value using the 1+2i syntax, e.g., z <- 3 + 4i. Alternatively, use the complex() function, e.g., complex(real = 3, imaginary = 4).

32. Find the factorial of 6. Find the mean, median, standard deviation of 2,4,9,10,14,28,52.

Code for factorial of 6:

```
facto<-function(a){
  if(a==1 | a==0){
    return(1)
  }else{
    return(a*facto(a-1))
  }
}


print(paste("The factorial of 6 is",facto(6)))
```

The output is:

```
> print(paste("The factorial of 6 is",facto(6)))
[1] "The factorial of 6 is 720"
>
```

Code for Finding the mean, median, standard deviation of 2,4,9,10,14,28,52.:

```
x<-c(2,4,9,10,14,28,52)

m=mean(x)
med=median(x)
standard_dev=sd(x)

print(paste("the mean is ",m))
print(paste("the median is ",med))
print(paste("the standard deviation is ",standard_dev))
```

The output is:

```
> print(paste("the mean is ",m))
[1] "the mean is  17"
> print(paste("the median is ",med))
[1] "the median is  10"
> print(paste("the standard deviation is ",standard_dev))
[1] "the standard deviation is  17.6162803489651"
>
```

References:

Allaire, J., Xie, Y., McPherson, J., Luraschi, J., Ushey, K., Atkins, A., ... & Cheng, J. (2022). rmarkdown: Dynamic Documents for R. R package version 2.14.

Xie, Y. (2015). Dynamic Documents with R and knitr. Chapman and Hall/CRC.